Tecnicatura Universitaria en Programación

Trabajo Práctico N° 9: Food Store en MongoDB

Bases de Datos II - Unidad 7: NoSQL y nuevas formas de almacenamiento

Integrantes: Facundo Deseff, Santino Godoy, Facundo Ramírez y Matías Vargas
Repositorio: github.com/FacundoDeseff/Tps_BasedeDatos
Fecha: 29/10/2026

Introducción

En este trabajo pasamos una parte de la base de Food Store a MongoDB. Tomamos los pedidos con sus líneas y los datos básicos del cliente, ejecutamos cuatro consultas y las comparamos con PostgreSQL. También le pedimos a una IA que comparara los dos motores y después revisamos dos de sus afirmaciones con la documentación y con pruebas en el laboratorio.

1. Modelo en MongoDB

1.1 Qué parte de la base tomamos

De la base relacional tomamos las tablas `pedido`, `detalle_pedido`, `usuario`, `producto` y `categoria`. La idea fue guardar el pedido completo en un solo documento, porque es lo que más se consulta en Food Store.

Tabla	Clave primaria	Relación	Uso en MongoDB
`pedido`	`id_pedido`	`usuario_id → usuario.id_usuario`	Documento principal
`detalle_pedido`	`id_detalle`	`pedido_id → pedido; producto_id → producto`	Líneas dentro de `items[]`
`usuario`	`id_usuario`	—	Datos básicos en `cliente`
`producto / categoria`	`id_producto / id_categoria`	`producto.id_categoria → categoria`	Referencia y nombre en cada línea

1.2 Cómo armamos el documento

Nos pareció que el pedido era la unidad natural. Las líneas van dentro de un arreglo `items[]` porque siempre se leen con la cabecera, son pocas y no tienen sentido sueltas. Así el pedido se lee de una sola vez y se inserta completo en una sola operación.

- **Cliente:** guardamos solo `id_usuario`, nombre y apellido. El resto del maestro queda afuera.
- **Producto:** guardamos `id_producto`, nombre, categoría y el precio que tenía en ese momento. No copiamos todo el catálogo porque el stock y el precio de lista cambian con el tiempo.
- **Detalles:** no hicimos una colección aparte. Sería lo mismo que la tabla relacional y nos obligaría a usar `$lookup` para armar un pedido.

Lo que aceptamos es que el nombre del cliente y del producto queden repetidos en cada pedido. Nos parece bien porque así el pedido guarda lo que pasó en ese momento y no cambia si después se modifica el maestro.

1.3 Campos de la colección `foodstore_nosql.pedidos`

Campo	Tipo	Qué guarda
`_id / id_pedido`	int	El id original del pedido. Se guarda dos veces para no perder la referencia.
`fecha`	date	La fecha del pedido, en la zona horaria de Buenos Aires.
`estado`	string	`PENDIENTE`, `EN_PREPARACION`, `ENTREGADO` o `CANCELADO`.
`total`	number	El total del pedido, copiado de `pedido.total`.
`cliente`	object	`id_usuario`, nombre y apellido.

Campo	Tipo	Qué guarda
`items[]`	array	`id_detalle`, `id_producto`, `nombre_producto`, `id_categoria`, `categoria`, `cantidad`, `precio_unitario_historico`, `subtotal` y `eliminado`.
`lineas_count`	int	Cuántas líneas tiene el pedido. Sirve para controlar.
`moneda / origen`	string / object	`ARS` y los datos de dónde vino el documento.
`migrado_en / schema_version`	date / int	Cuándo se migró y qué versión del modelo se usó.

1.4 Ejemplo de un documento

Este es el pedido 1 tal como quedó en MongoDB:

```
{
  "_id": 1, "id_pedido": 1, "fecha": ISODate("2026-09-29T13:30:21.757-03:00"),
  "estado": "ENTREGADO", "total": 4606.71,
  "cliente": { "id_usuario": 2578, "nombre": "Usuario_2578", "apellido": "Apellido_2578" },
  "items": [
    { "id_detalle": 1, "id_producto": 5829, "nombre_producto": "Producto_5829",
      "id_categoria": 7, "categoria": "Pastas", "cantidad": 2,
      "precio_unitario_historico": 1580.345, "subtotal": 3160.69, "eliminado": false },
    { "id_detalle": 2, "id_producto": 32828, "nombre_producto": "Producto_32828",
      "id_categoria": 7, "categoria": "Pastas", "cantidad": 2,
      "precio_unitario_historico": 723.01, "subtotal": 1446.02, "eliminado": false }
  ],
  "lineas_count": 2, "moneda": "ARS",
  "origen": { "sistema": "PostgreSQL", "base": "foodstore", "tabla": "pedido", "eliminado": false },
  "migrado_en": ISODate("2026-10-29T12:00:00-03:00"), "schema_version": 1
}
```

1.5 Estado y precio histórico

El estado no existía en PostgreSQL: la tabla `pedido` solo tiene `eliminado`. Para no inventar datos, lo tradujimos así: `eliminado = true` pasa a `CANCELADO` y `eliminado = false` pasa a `ENTREGADO`. El valor original queda en `origen.eliminado`. Los estados `PENDIENTE` y `EN_PREPARACION` quedan disponibles para pedidos nuevos.

El precio unitario tampoco estaba guardado. Lo calculamos como `subtotal / cantidad` y lo dejamos fijo en la línea. No usamos `producto.precio_lista` porque ese es el precio de hoy y no el de la venta.

2. Migración y reglas de integridad

Armamos la base `foodstore_nosql` y la colección `pedidos`. Le pusimos un validador `$jsonSchema` y creamos los índices. Pasamos 6 pedidos que ya existían en `foodstore` con un solo `insertMany()`: 5 `ENTREGADO` y 1 `CANCELADO` (el pedido 3, que tenía `eliminado = true`).

Antes y después de insertar, el script revisa que los totales y las líneas estén bien. Si algo no cierra, corta la migración.

```
Documentos insertados: 6
Integridad OK
ENTREGADO: 5
CANCELADO: 1

Pedidos revisados: 6
Pedidos con total != suma de subtotales: 0
Líneas con subtotal != cantidad × precio: 0
```

Las dos reglas que nos pidieron conservar fueron estas:

- **Regla 1, identificadores:** `_id` e `id_pedido` guardan el `id_pedido` original; cada línea guarda `id_detalle` e `id_producto`; el cliente guarda `id_usuario`. Además, `items` no puede quedar vacío y hay índices únicos para `id_pedido` e `items.id_detalle`.

- **Regla 2, importes:** `total` tiene que ser igual a la suma de los `subtotal` y cada `subtotal` tiene que dar `cantidad × precio_unitario_historico`. Eso lo controla el script porque `$jsonSchema` no puede comparar dos campos entre sí.

Los índices que creamos son: `_id`, `id_pedido` (único), `estado`, `cliente.id_usuario`, `items.id_producto`, `items.id_detalle` (único), `total` y `fecha` (descendente). Los usan las consultas de la parte 2.

3. Consultas en MongoDB

Ejecutamos estas cuatro consultas en `mongosh`:

#	Consulta	Cantidad	IDs
a	Todos los pedidos, mostrando `_id` y `estado`	6	1, 2, 3, 4, 5, 6
b	Pedidos con `estado: "ENTREGADO"`	5	1, 2, 4, 5, 6
c	Pedidos que tienen el producto `5829` en `items[]`	1	1
d	Pedidos con `total > 5000`	3	2, 4, 5

```
db.pedidos.find({}, { _id: 1, estado: 1 })
db.pedidos.find({ estado: "ENTREGADO" })
db.pedidos.find({ "items.id_producto": 5829 })
db.pedidos.find({ total: { $gt: 5000 } })
```

El umbral de 5000 lo elegimos porque los totales van de 3283,34 a 7208,54 y el promedio es 4949,64. Con 5000 quedan 3 pedidos grandes (2, 4 y 5) y 3 chicos (1, 3 y 6).

En la consulta c) usamos notación de punto porque hay una sola condición sobre el arreglo. Si tuviéramos que combinar dos condiciones de la misma línea, haría falta `$elemMatch`.

4. Comparación con PostgreSQL

Para comparar, hicimos las consultas b y c en PostgreSQL y las limitamos a los pedidos 1 a 6, que son los que pasamos a MongoDB. Si no las limitábamos, la comparación no tenía sentido porque `foodstore` tiene 200.000 pedidos.

Como `pedido` no tiene estado, `ENTREGADO` lo buscamos con `eliminado = false`. Para el producto usamos `EXISTS`, así no se duplica el pedido si el producto aparece en dos líneas.

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los IDs?
Estado `ENTREGADO`	5 (1, 2, 4, 5, 6)	5 (1, 2, 4, 5, 6)	Sí
Producto `5829`	1 (1)	1 (1)	Sí

En los dos casos coincidieron la cantidad y los IDs. La diferencia más grande aparece si no limitamos el universo: PostgreSQL devuelve 194094 pedidos `ENTREGADO` y 3 pedidos con el producto 5829 (1, 33138 y 132374). Los últimos dos no los migramos, por eso no aparecen en MongoDB. No es un error del modelo, es que la colección tiene solo 6 pedidos.

El pedido 3 queda afuera de `ENTREGADO` en los dos motores porque tiene `eliminado = true`. Las líneas eliminadas no las filtramos porque en el origen no hay ninguna con `eliminado = true`. No tuvimos que cambiar el modelo ni las consultas.

5. Uso de IA

Le pedimos a opencode, con el modelo `deepseek-v4.1-flash-free`, que comparara PostgreSQL y MongoDB para este caso. En la consulta le aclaramos el modelo actual, que el acceso más común es leer el pedido completo y que había que cuidar la coherencia de cantidades y precios.

```
Compará PostgreSQL y MongoDB para almacenar y consultar los pedidos de Food Store. El modelo
actual es relacional: pedido, detalle_pedido, usuario, producto y categoria, con claves foráneas y
tipos NUMERIC(12,2) para los importes. El acceso más frecuente es leer el pedido completo
(cabecera, cliente y líneas). Se necesita conservar la coherencia de cantidades y precios:
total = suma de subtotales, subtotal = cantidad × precio histórico y cantidad > 0. Indicá ventajas y
limitaciones de cada motor y si conviene mantener PostgreSQL, migrar ese subconjunto a MongoDB
o combinar ambos.
```

La IA dijo, en resumen, que MongoDB tiene ventaja para leer el pedido completo porque no necesita `JOIN` ni `$lookup`, y que PostgreSQL da más garantías por las claves foráneas, los `CHECK` y las transacciones ACID. También dijo que MongoDB no puede garantizar `total = suma de subtotales`. Al final recomendó combinar los dos motores.

5.1 Contraste de dos afirmaciones

Afirmación de la IA	Qué revisamos	Resultado
"Al embeber las líneas en `items[]`, un pedido completo se recupera con una sola lectura, sin `JOIN` ni `$lookup`."	La documentación de MongoDB recomienda embeber cuando los datos se consultan juntos. En `mongosh`, `findOne({ _id: 1 })` nos devolvió un documento con la cabecera, el cliente y las 2 líneas. La migración insertó los 6 pedidos con un solo `insertMany()`.	La parte de la lectura es cierta. No medimos velocidad, así que no podemos decir que sea más rápido.
"PostgreSQL garantiza **mejor** la coherencia de cantidades y precios porque es relacional […] MongoDB, en cambio, no puede garantizar que `total = suma de subtotales`."	`$jsonSchema` no compara campos entre sí. PostgreSQL tampoco puede hacer esa comparación con un `CHECK`, porque no puede mirar otras filas: necesita un trigger. En la prueba, los dos motores aceptaron un total de 1,00 con una línea de 100,00. Lo que sí funcionó fue la FK (rechazó un detalle huérfano) y los `CHECK` (rechazaron cantidad 0 y subtotal -10).	Es parcial. Es verdad que PostgreSQL controla mejor las referencias y los rangos, pero no garantiza la igualdad del total solo con restricciones. Los dos motores aceptaron el mismo error.

5.2 Conclusión

Para nosotros, lo mejor es combinar los dos motores. PostgreSQL tiene que seguir siendo la base principal: ya tiene las 200.000 órdenes, controla las claves foráneas, los tipos `NUMERIC`, las cantidades positivas y las transacciones ACID. MongoDB nos sirve como una copia de lectura de los pedidos completos: al tener el cliente y los ítems en un mismo documento, se lee todo de una vez y queda guardado el precio y el nombre del momento de la compra. La igualdad `total = suma de subtotales` no la garantiza ninguno de los dos solo con validaciones: hay que controlarla en la migración, en la aplicación o con un trigger. Pasar todo a MongoDB nos haría perder controles y tener que rehacerlos; quedarnos solo con PostgreSQL nos hace perder la lectura armada. Por eso proponemos PostgreSQL como sistema de registro y MongoDB como copia de lectura que se sincroniza y se verifica. Antes de ampliar el subconjunto habría que medir latencias y volúmenes reales.

6. Evidencias

Estos son los 6 pedidos que migramos:

id_pedido	Cliente	Estado	Total	En PostgreSQL
1	Usuario_2578	ENTREGADO	4606,71	eliminado = false
2	Usuario_7439	ENTREGADO	7208,54	eliminado = false
3	Usuario_3430	CANCELADO	3885,17	eliminado = true
4	Usuario_352	ENTREGADO	5515,37	eliminado = false
5	Usuario_7773	ENTREGADO	5198,69	eliminado = false
6	Usuario_6009	ENTREGADO	3283,34	eliminado = false

6.1 Salidas de las consultas

```
MongoDB
a) cantidad = 6  ids = [1, 2, 3, 4, 5, 6]
b) ENTREGADO  cantidad = 5  ids = [1, 2, 4, 5, 6]
c) producto 5829  cantidad = 1  ids = [1]
d) total > 5000  cantidad = 3  ids = [2, 4, 5]

PostgreSQL (acotado a los pedidos 1 a 6)
b) ENTREGADO  cantidad = 5  ids = {1, 2, 4, 5, 6}
c) producto 5829  cantidad = 1  ids = {1}

PostgreSQL sin acotar
ENTREGADO = 194094
producto 5829 = 3 (1, 33138, 132374)
```

6.2 Pruebas de integridad

Prueba	MongoDB	PostgreSQL
`total = 1,00` con una línea de `subtotal = 100,00`	Aceptado	Aceptado (con `ROLLBACK`)
Documento sin `items`	Rechazado por `$jsonSchema`	No aplica
`id_detalle` duplicado	Rechazado (E11000, índice único)	No aplica
Detalle con `pedido_id` inexistente	No aplica	Rechazado por FK
`cantidad = 0`	No lo probamos	Rechazado por `CHECK`
`subtotal = -10,00`	No lo probamos	Rechazado por `CHECK`

> MongoDB controla la estructura, los tipos y la unicidad, pero no la igualdad entre campos ni que las claves referenciadas existan. PostgreSQL controla las referencias, los tipos y los rangos con FK y `CHECK`, y permite atomicidad con ACID, pero tampoco compara `total` con la suma de subtotales sin un trigger. Los dos motores aceptaron el mismo documento incoherente.

Las pruebas completas están en `docs/p3_evidencia_mongodb.txt` y `docs/p3_evidencia_postgresql.txt`. Los resultados de las consultas están en `05_comparacion_resultados.md`. Los scripts adjuntos son `consultas_mongodb.js`, `consultas_postgresql.sql` y `03_migracion_pedidos.js`.